Stale-While-Revalidate

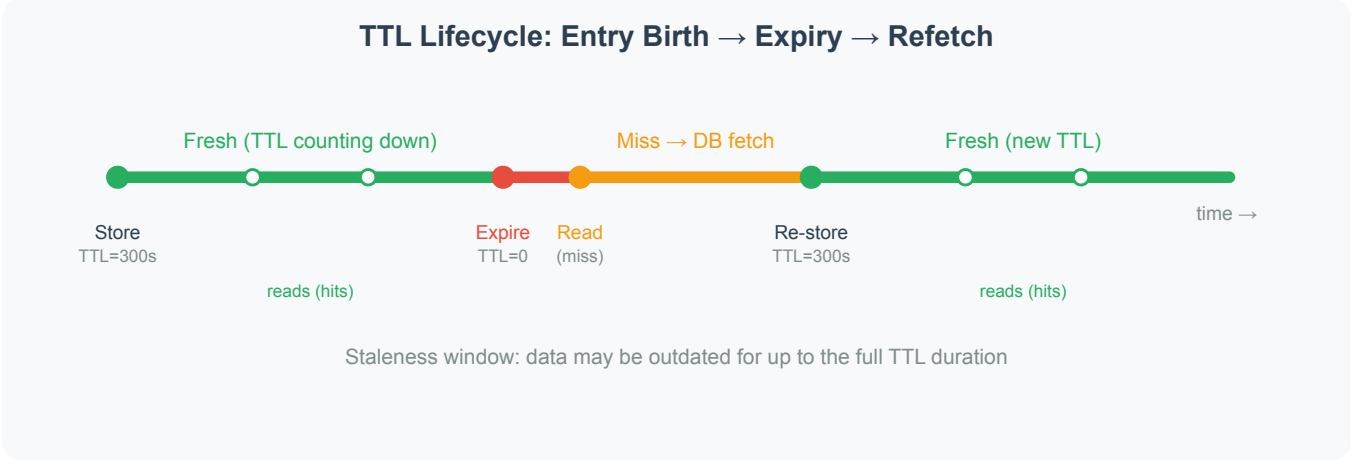
systemdesignschool.io/fundamentals/cache-invalidation

Invalidation & Freshness

The consistency article showed that caches can hold stale data. Invalidation is the mechanism that removes or refreshes stale entries. Get it wrong and users see outdated data. Get it too aggressive and the cache provides no benefit—every request falls through to the database.

Time-To-Live (TTL)

A TTL attaches a countdown to each cache entry. When the timer expires, the entry is automatically removed. The next read triggers a cache miss, fetches fresh data from the database, and stores it with a new TTL.



TTL is the simplest form of invalidation. It requires no coordination between services—each entry self-destructs on schedule. The tradeoff is staleness: data can be outdated for up to the full TTL duration before refreshing.

Choosing a TTL value depends on how stale the data can be without harming the user experience.

Data Type	Typical TTL	Reasoning
Session tokens	30 minutes	Match session timeout policy
Product catalog	5–60 minutes	Prices change infrequently but must update same-day
User profile	5–15 minutes	Changes are rare; brief staleness is acceptable
Search results	1–5 minutes	Results change frequently; short freshness window
Configuration	Hours to days	Rarely changes; long cache life saves load
Feature flags	30–60 seconds	Changes should propagate quickly across instances

Setting TTL too short defeats the purpose of caching—the database absorbs nearly full load. Setting it too long means users see outdated data. Most teams start with a moderate value (5–15 minutes) and adjust based on observed staleness complaints and cache hit rates.

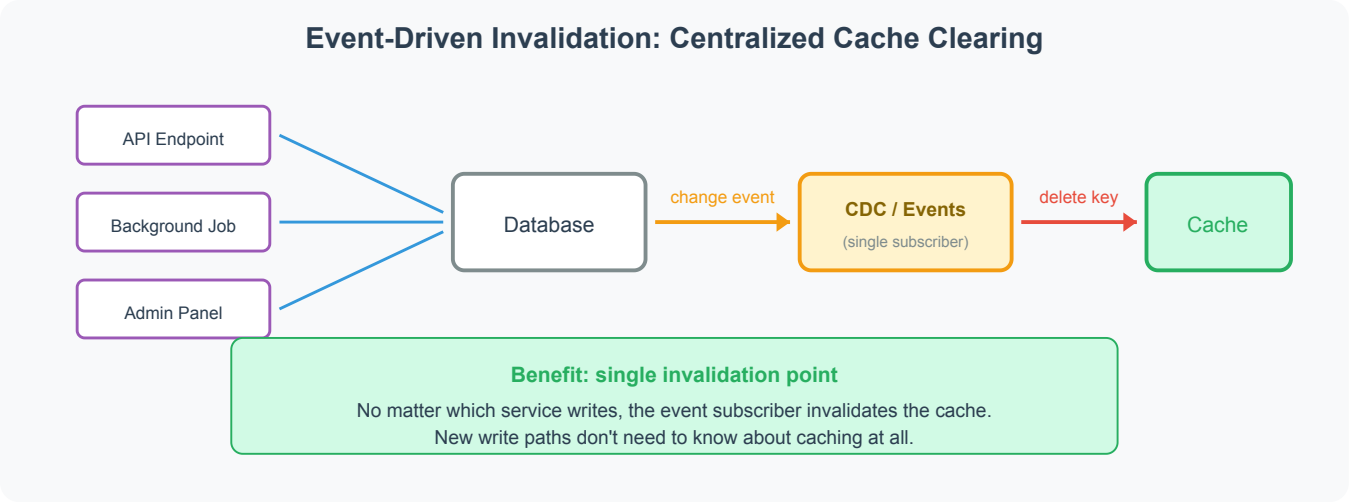
Explicit Invalidation

TTL is passive—it waits for time to pass. Explicit invalidation is active—the application deletes or overwrites a cache entry immediately when the underlying data changes. This is the "delete key" step in the write-around pattern.

Explicit invalidation provides tighter freshness than TTL alone. If the delete is synchronous and race-free, staleness is limited to the brief window between the database write and the cache delete.

The challenge is ensuring every write path triggers invalidation. If the application updates user data through an API endpoint, a background job, and an admin panel, all three must delete the cache key. Missing one path creates a silent staleness bug that's hard to diagnose.

Event-driven invalidation simplifies this. Instead of each writer knowing about the cache, the database emits change events (via change data capture or application-level events). A subscriber listens for changes and invalidates the corresponding cache keys. This centralizes invalidation logic in one place.



TTL as a Safety Net

Production systems combine both approaches. Explicit invalidation handles the normal case—data changes, cache updates immediately. TTL acts as a safety net—if invalidation fails (message lost, subscriber crashes, key name mismatch), the stale entry self-destructs after the TTL window.

This layered approach means the worst case is bounded staleness (the TTL duration) rather than unbounded staleness (stale forever until the next write).

Standard TTL creates a binary state: the entry is either fresh (return it) or expired (cache miss, fetch from database). The moment of expiration causes a latency spike—the requesting user waits for the database round trip.

Stale-while-revalidate softens this edge. When an entry expires, the cache returns the stale value immediately and triggers a background refresh. The user gets fast (slightly stale) data. Once the background refresh completes, subsequent requests get fresh data.

This pattern largely avoids the latency spike at expiration. The tradeoff is that one request after expiration receives stale data. For most use cases (product pages, search results, recommendations), this brief staleness is invisible to the user.

Some HTTP caches and CDNs support this via the `stale-while-revalidate` directive (RFC 5861) in `Cache-Control` headers. Application-level caches require custom logic: check if the entry is past its soft TTL, return it anyway, and spawn an async task to refresh.

TTL Jitter

When many entries share the same TTL (common when a batch of data is cached simultaneously), they all expire at the same moment. This creates a **synchronized miss storm**—thousands of requests hit the database at once, overwhelming it.

Adding jitter spreads expirations randomly. Instead of a fixed 300-second TTL, each entry gets a TTL between 270 and 330 seconds (300 ± 10%). Expirations distribute evenly across the window rather than clustering at one instant.

This is the same technique used to prevent the thundering herd problem discussed in the failure modes article later in this section.

From here, eviction policies decide which entries to sacrifice when the cache is full and no entries have expired yet.